DevOps for Cloud-Native Software Engineering

CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)

NORTHEASTERN.

Introduction to DevOps

Template Instructions

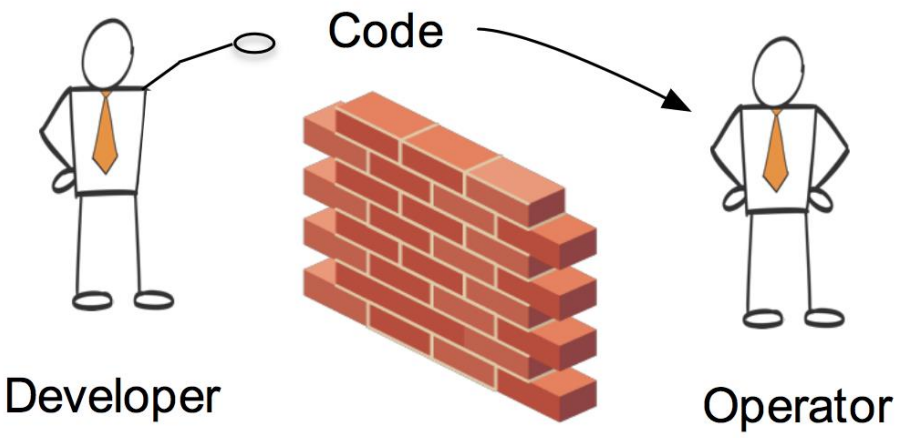
WORKED FINE IN DEV
OPS PROBLEM NOW

What is DevOps?

- DevOps is a set of practices, cultural philosophies, and tools that increase an organization's ability to deliver applications and services at high velocity. The term combines "Development" and "Operations," reflecting the integration of these traditionally separate functions.
- Core idea: Break down silos between teams who write code and teams who deploy and maintain it.

Historical Context

- Before DevOps, organizations typically operated with distinct development and operations teams.
- Developers would write code and "throw it over the wall" to operations for deployment. This created several problems: slow release cycles, finger-pointing when issues arose, and a disconnect between what developers built and how systems actually ran in production.
- The DevOps movement emerged in the late 2000s as a response to these challenges, influenced by Agile methodologies and the growing need for faster software delivery.



DevOps, SDLC, and Twelve-Factors

- **Relationship to SDLC:** DevOps doesn't replace the SDLC but rather accelerates and automates many of its phases. Where traditional SDLC might have discrete handoffs between phases, DevOps emphasizes continuous flow.
- **Relationship to Twelve-Factor App:** The twelve factors provide architectural guidelines that make DevOps practices possible. For example:
  - Factor V (Build, release, run) aligns directly with CI/CD pipelines
  - Factor X (Dev/prod parity) enables reliable deployments
  - Factor XI (Logs as event streams) supports observability
  - Factor III (Config in environment) enables environment-specific deployments without code changes

The Three Ways of DevOps

Template Instructions

Flow/Systems Thinking

- Focus on the performance of the entire system rather than individual silos.
- Work should flow smoothly from development through operations to the customer.
- Key practices:
  - Make work visible (Kanban boards, work-in-progress limits)
  - Reduce batch sizes and intervals between deployments
  - Build quality in rather than inspecting it in later
  - Eliminate waste and bottlenecks

Feedback Loops

- Create fast, constant feedback from right to left at all stages. When problems occur, detect them quickly and fix them while context is fresh.
- Key practices:
  - Automated testing at every stage
  - Monitoring and alerting in production
  - Telemetry and observability
  - Blameless post-mortems

Continuous Learning and Experimentation

- Foster a culture of continuous experimentation, learning from failures, and understanding that repetition and practice lead to mastery.
- Key practices:
  - Allocate time for improvement and innovation
  - Create shared repositories of knowledge
  - Inject failures to build resilience (chaos engineering)
  - Institutionalize improvement of daily work

DevOps Workflow

https://resources.github.com/devops/fundamentals/

Core DevOps Practices

Template Instructions

DevOps Practices

- Communication & Collaboration
- Infrastructure as Code
- Continuous Integration, Continuous Delivery, Continuous Deployment
- Version Control
- Automation
- Logging and Monitoring
- Security

Communication & Collaboration

- The key premise behind DevOps is collaboration.
- Development and operations teams coalesce into a functional team that communicates, shares feedback, and collaborates throughout the entire development and deployment cycle.

Infrastructure as Code

- Traditionally, infrastructure is provisioned using manual process.
- A fundamental principle of DevOps is to treat infrastructure the same way developers treat code.
- Practicing "Infrastructure as Code" means applying the same rigor of application code development to infrastructure provisioning and setup.
- All infrastructure provisioning "code" and environment configuration must be stored in version management system such as Git.
- Same programming best practices must apply to the infrastructure code as applied to application code.
- Infrastructure provisioning, orchestration, and deployment should support the use of "infrastructure code".

Automation

- Another core philosophy and practice of DevOps is automation.
- Automation focuses on the setup, configuration, deployment, and support of infrastructure and the applications that run on it.
- By using automation, you can set up environments more rapidly in a standardized and repeatable manner. The removal of manual process is a key to a successful DevOps strategy.
- Historically, server configuration and application deployment has been a predominantly a manual process. Environments become nonstandard, and reproducing an environment when issue arises is difficult.

Benefits of Automation

- Rapid Changes
- Improved Productivity
- Repeatable Configurations
- Leveraged Elasticity
- Leveraged auto scaling
- Automated Testing

@DEVOPS_BORAT
DevOps Borat

To make error is human. To propagate error to all server in automatic way is #devops.

Continuous Integration

- Continuous integration is a software development practice where developers regularly merge their code changes into a central repository, after which automated builds and tests are run.
- The key goals of continuous integration are to find and address bugs quicker, improve software quality, and reduce the time it takes to validate and release new software updates.

Continuous Delivery & Continuous Deployment

- Continuous delivery is a software development practice where code changes are automatically built, tested, and prepared for a release to production but not deployed without some manual intervention.
- Continuous deployment is a software development practice where code changes are automatically built, tested, and deployed to production.

CI/CD PIPELINE

TEST → BUILD → DEPLOY DEV → MANUAL TWEAKS → FORGET ABOUT THE TWEAKS → DEPLOY STAGING → NOTHING WORKS → MORE TWEAKS → WAIT FOR CHANGE APPROVAL

WAIT FOR CHANGE APPROVAL → HEAT DEATH OF UNIVERSE → ESCALATE TO DIRECTOR → CHANGE APPROVED SIGHT-UNSEEN → WAIT FOR 3AM MAINTENANCE WINDOW → DEPLOY TO PROD → EVERYTHING WORKS!

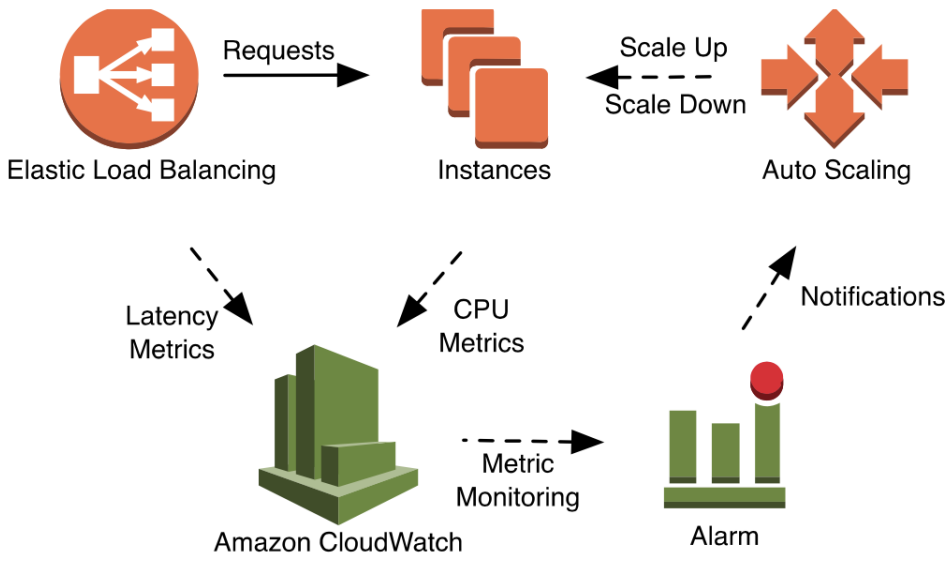
EVERYTHING WORKS! → JUST KIDDING. WHY WOULD ANY OF THIS WORK → ERRORS / DOWNTIME / PANIC → 50-PERSON CONFERENCE CALL → MANY MANY MANUAL TWEAKS → PROD RETURNS TO AN ACCEPTABLY BROKEN STATE

ADD LINKEDIN SKILLS: DEVOPS, AUTOMATION, TROUBLESHOOTING

@forrestbrazeal

Monitoring

- Communication and collaboration is fundamental in a DevOps strategy.
- To facilitate this, feedback is critical.
- Feedback comes from logs, monitoring, alerting and auditing infrastructure so developers and operations teams can work together closely and transparently.



Security

- In a DevOps enabled environment, focus on security is still of paramount importance.
- Infrastructure and company assets needs to be protected, and when issue arise, they need to be rapidly and effectively addressed.

Adopting DevOps Philosophy

Template Instructions

Getting Started

Organizations new to DevOps should focus on foundational practices:

- Version control everything (code, configuration, infrastructure definitions)
- Automate builds and basic testing
- Make deployments repeatable and scripted
- Implement basic monitoring and alerting
- Conduct blameless post-mortems after incidents

Common Challenges

- **Legacy systems:** Older applications may not fit cloud-native patterns. Consider strangler fig pattern for gradual modernization.
- **Organizational resistance:** Cultural change is often harder than technical change. Start with willing teams and demonstrate value.
- **Tool sprawl:** The DevOps ecosystem includes countless tools. Focus on solving problems rather than adopting tools for their own sake.
- **Over-automation:** Not everything needs to be automated immediately. Prioritize automation that reduces the most toil or risk.

Measuring DevOps Success

The DORA (DevOps Research and Assessment) metrics provide a framework for measuring DevOps performance:

- **Deployment frequency:** How often does your organization deploy to production?
- **Lead time for changes:** How long does it take from code commit to running in production?
- **Change failure rate:** What percentage of deployments cause failures requiring remediation?
- **Time to restore service:** How long does it take to recover from a failure?

Key Takeaways

Template Instructions

Summary

- DevOps is about culture and collaboration as much as tools and automation.
- The Three Ways provide a mental model: optimize flow, amplify feedback, enable continuous learning.
- CI/CD pipelines automate the path from code to production, catching issues early and enabling frequent, reliable releases.
- Infrastructure as Code brings software engineering practices to infrastructure management.
- Observability (metrics, logs, traces) provides the visibility needed to operate complex distributed systems.
- Security must be integrated throughout the development lifecycle, not bolted on at the end.
- The twelve-factor methodology provides architectural patterns that enable effective DevOps practices.

Recommended Reading

Template Instructions

Resources

- **Books:**
  - "The Phoenix Project" by Gene Kim, Kevin Behr, and George Spafford
  - "The DevOps Handbook" by Gene Kim, Jez Humble, Patrick Debois, and John Willis
  - "Accelerate" by Nicole Forsgren, Jez Humble, and Gene Kim
  - "Site Reliability Engineering" by Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Murphy (available free online from Google)
- **Online Resources:**
  - DORA State of DevOps Reports
  - Martin Fowler's website (articles on CI/CD, microservices)
  - The Twelve-Factor App (12factor.net)